

# Отчет по Практической работе №4 факультатива “Введение в GameDev: основы игрового ИИ”

Тема: Эволюционные и роевые алгоритмы

Подготовила: Сусликова Вероника Денисовна

Номер ИСУ: 467632

Поток: 1.1

---

## Задание 3: Мини-реферат. Алгоритмы, вдохновленные природой

“Искусственные иммунные системы”

### Биологические Предпосылки:

Искусственные иммунные системы (Artificial Immune Systems, AIS, ИИС) — это класс алгоритмов, вдохновленных процессами, происходящими в иммунной системе человека. Ключевые биологические компоненты и понятия, лежащие в основе ИИС:

- Антигены: Молекулы, распознаваемые иммунной системой как чужеродные или опасные. В ИИС антигены представляют собой данные, которые необходимо обнаружить, классифицировать или оптимизировать (при решении задач распознавания образов, вне нее - просто некие данные).
- Антитела: Молекулы, связывающиеся с антигенами, нейтрализуя их или маркируя для уничтожения. В ИИС антитела представляют собой возможные решения проблемы или (вне контекста задач распознавания образов) детекторы данных.
- Лимфоциты: Клетки иммунной системы, включая В-клетки и Т-клетки, отвечающие за распознавание антигенов и выработку антител. В ИИС лимфоциты представляют собой вычислительные агенты, работающие над поиском и улучшением решений.
- Клональный отбор: Процесс, в котором лимфоциты, распознавшие антиген, клонируются (размножаются), и их антитела подвергаются соматической гипермутации (случайным изменениям), что повышает их аффинность (способность связываться с антигеном). В ИИС клональный отбор позволяет улучшать и адаптировать существующие решения. Аффинность связей антител друг с другом определяем как расстояние между соответствующими векторами. Аналогично определяем аффинность связей антитела и антигена. Полагаем, что меньшим значениям расстояния соответствует большая аффинность связей между агентами.

### Этапы Алгоритма:

- 1) Инициализируем популяции антител и антигенов.
- 2) Для каждого из антигенов выполняем следующие действия:

- Вычисляем аффинность антигена со всеми антителами иммунной системы (сети) и отбираем среди указанных антител  $N$  агентов с максимальной аффинностью.
- Клонировем отобранные антитела с коэффициентом клонирования, пропорциональным их аффинности, так чтобы общее число клонов было равно  $N \cdot C$  где  $C$  - число клонов, создаваемых каждым из отобранных антител. Все полученные клоны подвергаем мутации с степенью, определенной таким образом, чтобы сила мутации большей группы клонов была ниже, чем у меньшей.
- Вычисляем для полученных антител их аффинность с антигеном. Среди всех полученных антител выбираем  $Q = S \cdot N \cdot C$  антител (где  $S$  - степень селекции, то есть относительное число улучшенных антител, отбираемых из множества клонированных клеток), имеющих максимальную аффинность, и помещаем в популяцию клеток памяти иммунной сети (являющуюся подмножеством популяции антител). Удаляем из популяции клеток памяти иммунной сети те клетки, аффинность которых ниже величины  $P$  - заранее определенного порогового коэффициента гибели или стимуляции антител в зависимости от их аффинности.
- Вычисляем аффинность всех клеток скорректированной популяции клеток памяти иммунной сети. Удаляем из популяции тех агентов, аффинность которых ниже заранее заданной пороговой величины сжатия сети (клональное сжатие). Расширяем популяцию антител агентами по популяции клеток памяти иммунной сети.

3) Вычисляем аффинность агентов расширенной популяции и удаляем из этой сети тех агентов, аффинность которых ниже заранее заданной пороговой величины сжатия сети (сетевое сжатие).

4) Заменяем  $B$  процентов худших (с точки зрения аффинности) агентов популяции новыми, случайно сгенерированными клетками (где  $B$  - заранее установленный коэффициент обновления иммунной сети.).

5) Проверяем выполнение условий окончания итераций (остановы) и в зависимости от результата проверки либо переходим к шагу 2, либо завершаем вычисления.

### Пример одной итерации:

Имеем популяцию антител  $Ab$  размера  $N = 5$ :

- $Ab_1: x = -2.5, f(Ab_1) = 6.81$  (аффинность)
- $Ab_2: x = 1.0, f(Ab_2) = 1.84$  (аффинность)
- $Ab_3: x = 3.0, f(Ab_3) = 9.85$  (аффинность)
- $Ab_4: x = -0.5, f(Ab_4) = 0.04$  (аффинность)
- $Ab_5: x = 4.0, f(Ab_5) = 16.76$  (аффинность)

Выбираем  $N = 3$  лучших антитела на основе их аффинности (чем больше значение, тем лучше антитело)

Отобранные антитела:

- $Ab_5$ :  $x = 4.0$ ,  $f(Ab_5) = 16.76$  (лучшее)
- $Ab_3$ :  $x = 3.0$ ,  $f(Ab_3) = 9.85$
- $Ab_1$ :  $x = -2.5$ ,  $f(Ab_1) = 6.81$

Каждое из выбранных антител клонируется в количестве, пропорциональном их аффинности, вычисленном по определенной формуле. Для простоты предположим, что количество клонов для каждого антитела следующее:

- $Ab_5$ : 4 клона,  $Ab_3$ : 2 клона,  $Ab_1$ : 1 клон

Применяем мутацию к клонам. Степень мутации обратно пропорциональна аффинности родительского антитела (чем лучше родитель, тем меньше мутация). Мутация заключается в добавлении случайного значения из интервала от -степени мутации до степени мутации

- Клоны  $Ab_5$  (4 клона, affinity = 16.76):
  - $Ab_{5\_clone1}$ :  $x = 4.0 + \text{random}(-0.059, 0.059) \approx 4.02$
  - $Ab_{5\_clone2}$ :  $x = 4.0 + \text{random}(-0.059, 0.059) \approx 3.97$
  - $Ab_{5\_clone3}$ :  $x = 4.0 + \text{random}(-0.059, 0.059) \approx 4.04$
  - $Ab_{5\_clone4}$ :  $x = 4.0 + \text{random}(-0.059, 0.059) \approx 3.99$
- Клоны  $Ab_3$  (2 клона, affinity = 9.85):
  - $Ab_{3\_clone1}$ :  $x = 3.0 + \text{random}(-0.1, 0.1) \approx 3.08$
  - $Ab_{3\_clone2}$ :  $x = 3.0 + \text{random}(-0.1, 0.1) \approx 2.95$
- Клон  $Ab_1$  (1 клон, affinity = 6.81):
  - $Ab_{1\_clone1}$ :  $x = -2.5 + \text{random}(-0.15, 0.15) \approx -2.60$

Вычисляем аффинность мутировавших клонов (цифры условные):

- $f(Ab_{5\_clone1}) = f(4.02) \approx 16.96$
- $f(Ab_{5\_clone2}) = f(3.97) \approx 15.76$
- $f(Ab_{5\_clone3}) = f(4.04) \approx 17.32$
- $f(Ab_{5\_clone4}) = f(3.99) \approx 15.92$
- $f(Ab_{3\_clone1}) = f(3.08) \approx 9.49$
- $f(Ab_{3\_clone2}) = f(2.95) \approx 8.70$
- $f(Ab_{1\_clone1}) = f(-2.60) \approx 6.76$

Теперь у нас есть 7 мутировавших клонов и 5 исходных антител, всего 12 "особей". Из них нужно отобрать  $N = 5$  лучших для формирования следующей популяции. Для этого сортируем все особи по аффинности и выбираем 5 лучших:

1.  $Ab_{5\_clone3}$ :  $x = 4.04$ ,  $f(Ab_{5\_clone3}) \approx 17.32$
2.  $Ab_{5\_clone1}$ :  $x = 4.02$ ,  $f(Ab_{5\_clone1}) \approx 16.96$
3.  $Ab_5$ :  $x = 4.0$ ,  $f(Ab_5) = 16.76$
4.  $Ab_{5\_clone4}$ :  $x = 3.99$ ,  $f(Ab_{5\_clone4}) \approx 15.92$
5.  $Ab_{5\_clone2}$ :  $x = 3.97$ ,  $f(Ab_{5\_clone2}) \approx 15.76$

Новая популяция:

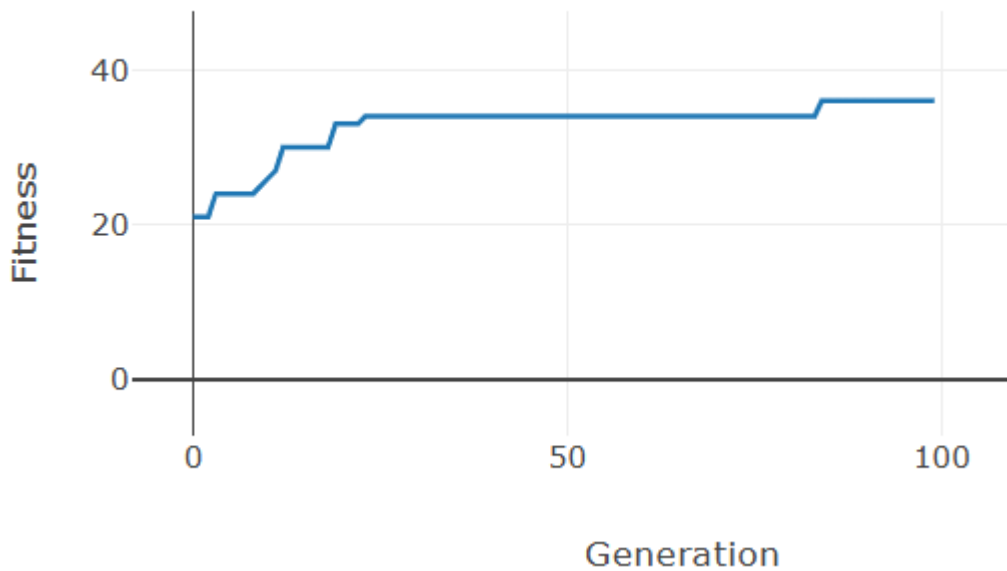
Новая популяция антител Ab размера N = 5:

- Ab<sub>1</sub>: x = 4.04, f(Ab<sub>1</sub>) ≈ 17.32
- Ab<sub>2</sub>: x = 4.02, f(Ab<sub>2</sub>) ≈ 16.96
- Ab<sub>3</sub>: x = 4.0, f(Ab<sub>3</sub>) = 16.76
- Ab<sub>4</sub>: x = 3.99, f(Ab<sub>4</sub>) ≈ 15.92
- Ab<sub>5</sub>: x = 3.97, f(Ab<sub>5</sub>) ≈ 15.76

## Задание 1: Генетический алгоритм

Задача о рюкзаке

Эксперимент 1



Gen. limit = 100, Mutation type = Flip-bit, Selection type = Elite selection

Crossing-over type: Single-point crossing over, Mutation probability = 50%

Результат: максимальное значение фитнес-функции равно 36 (выросло на 18 единиц)

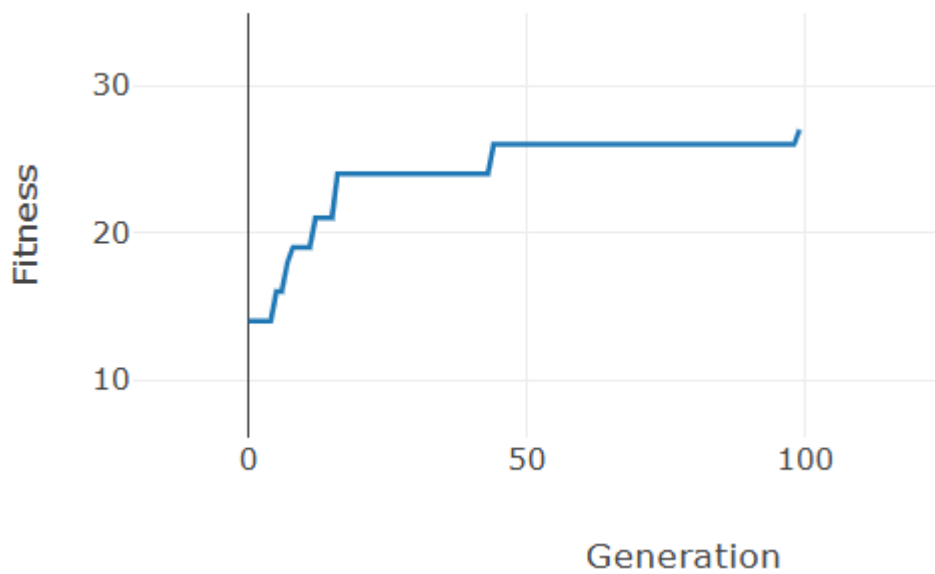
Эксперимент 2

Gen. limit = 100, Mutation type = Flip-bit, Selection type = Elite selection

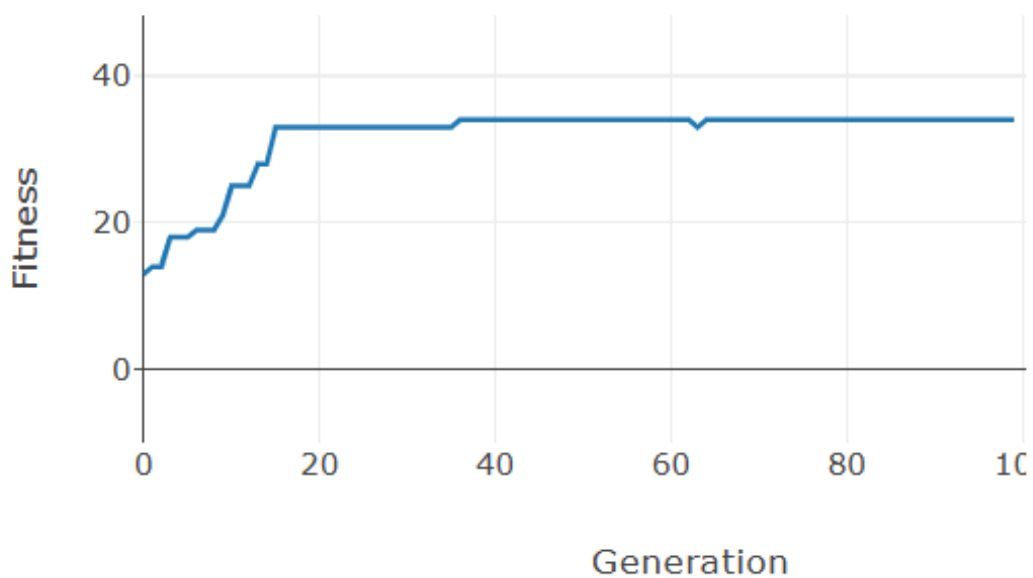
Crossing-over type: Single-point crossing over, Mutation probability = 25%

Результат: максимальное значение фитнес-функции равно 27 (выросло на 13 единиц)

Вывод: при уменьшении вероятности мутации развитие медленнее, так как шанс на изменения (положительные) соответственно снижается



### Эксперимент 3

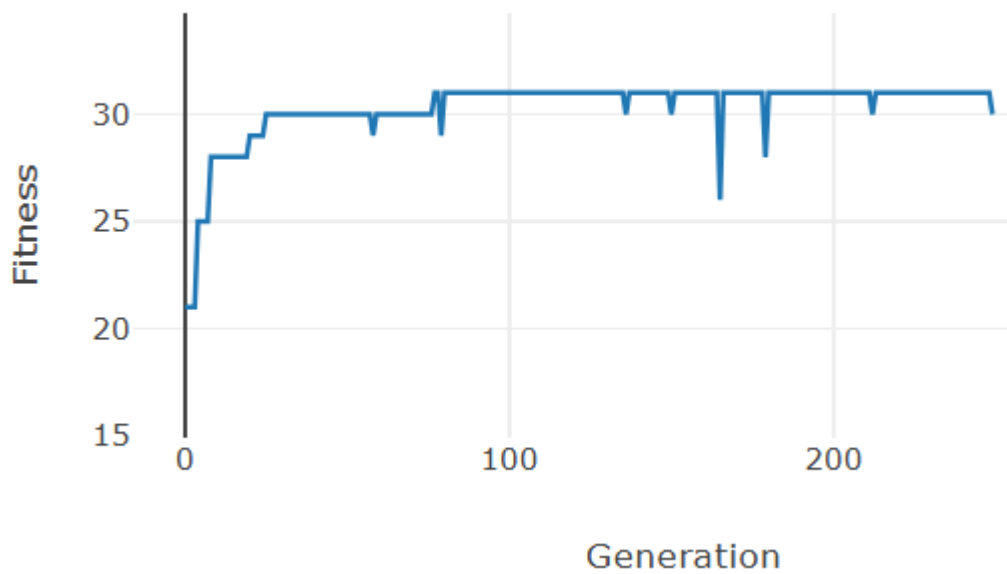


Gen. limit = 100, Mutation type = Flip-bit, Selection type = Elite selection  
 Crossing-over type: Single-point crossing over, Mutation probability = 75%

Результат: максимальное значение фитнес-функции равно 32 (выросло на 8 единиц)

Вывод: при увеличении вероятности мутации развитие может замедлиться, так как повышается и шанс на отрицательные изменения, что например произошло на 62 поколениях

### Эксперимент 4

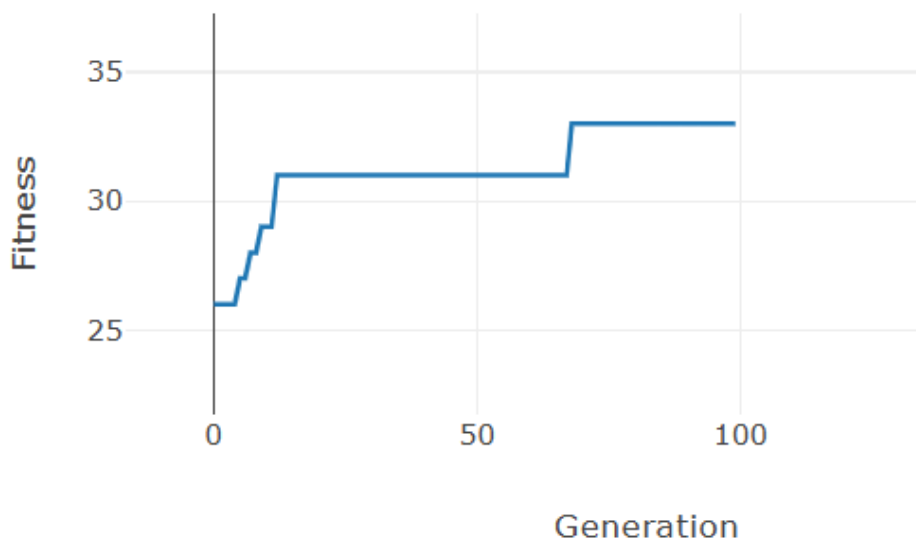


Gen. limit = 250, Mutation type = Flip-bit, Selection type = Elite selection  
 Crossing-over type: Single-point crossing over, Mutation probability = 100%

Результат: итоговое значение фитнес-функции равно 30, максимальное 31 (выросло на 9 единиц)

Вывод: при постоянной мутации развитие замедляется, так как шанс на отрицательные изменения (мутацию компонентов, которые и так имели хорошее качество и не должны мутировать) слишком высок, поколения тратятся на то, чтобы устранять последствия отрицательных изменений

#### Эксперимент 5

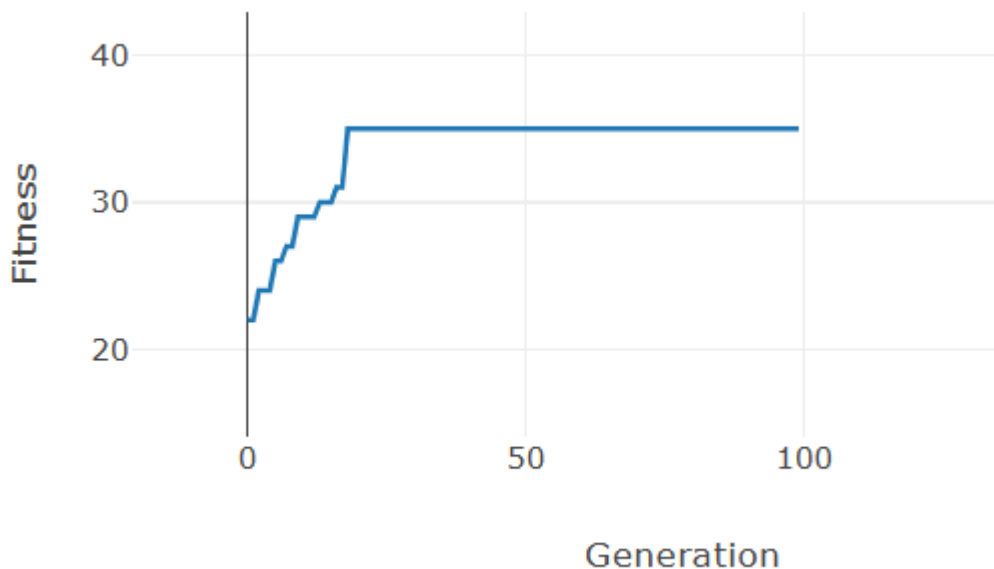


Gen. limit = 100, Mutation type = Inversion, Selection type = Elite selection  
 Crossing-over type: Single-point crossing over, Mutation probability = 50%

Результат: итоговое значение фитнес-функции равно 33 (выросло на 7 единиц)

Вывод: мутация инверсии менее эффективна чем Flip-bit, так как вносит более точные изменения, не способствующие быстрому активному росту

#### Эксперимент 6



Gen. limit = 100, Mutation type = Flip-bit, Selection type = Elite selection  
Crossing-over type: Uniform crossover, Mutation probability = 50%

Результат: итоговое значение фитнес-функции равно 35 (выросло на 13 единиц)

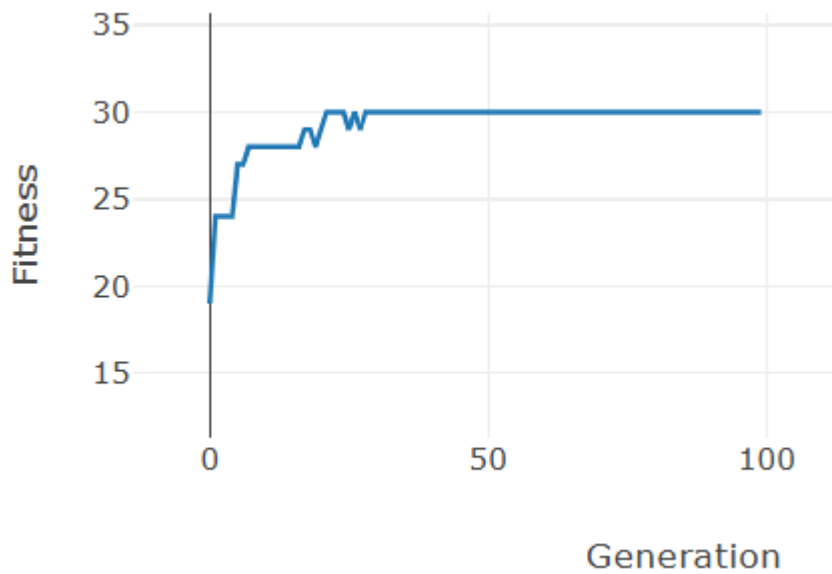
Вывод: Uniform crossover менее эффективен чем Single-point crossing over, имеет быстрое возрастание функции на первых поколениях, но потом возникает продолжительный период стагнации (отсутствия роста)

#### Эксперимент 7

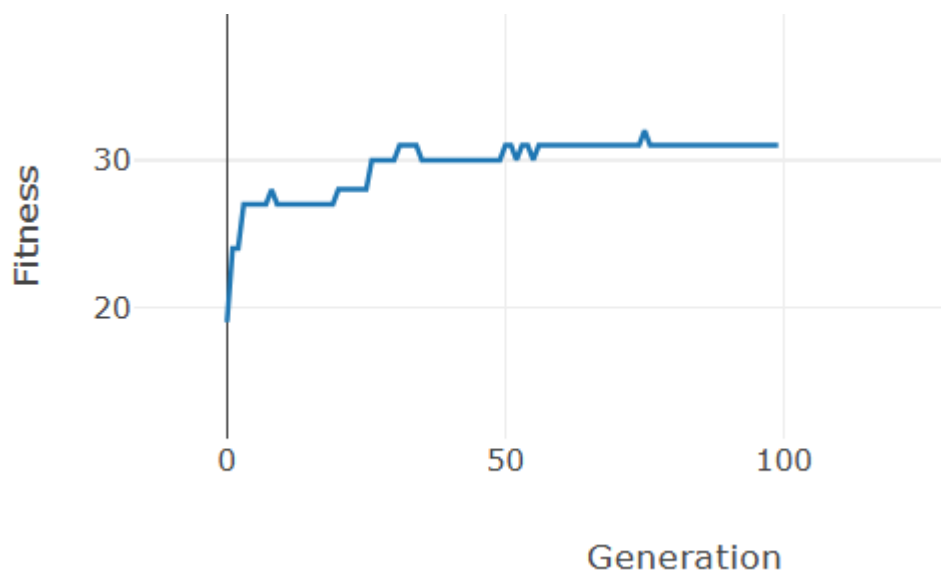
Gen. limit = 100, Mutation type = Flip-bit, Selection type = Elite selection  
Crossing-over type: Two-point crossover, Mutation probability = 50%

Результат: итоговое значение фитнес-функции равно 35 (выросло на 11 единиц)

Вывод: Two-point crossover менее эффективен чем Single-point crossing over, имеет быстрое возрастание функции на первых поколениях, но потом возникает продолжительный период стагнации (отсутствия роста), плюс возникают частые отрицательные изменения



#### Эксперимент 8

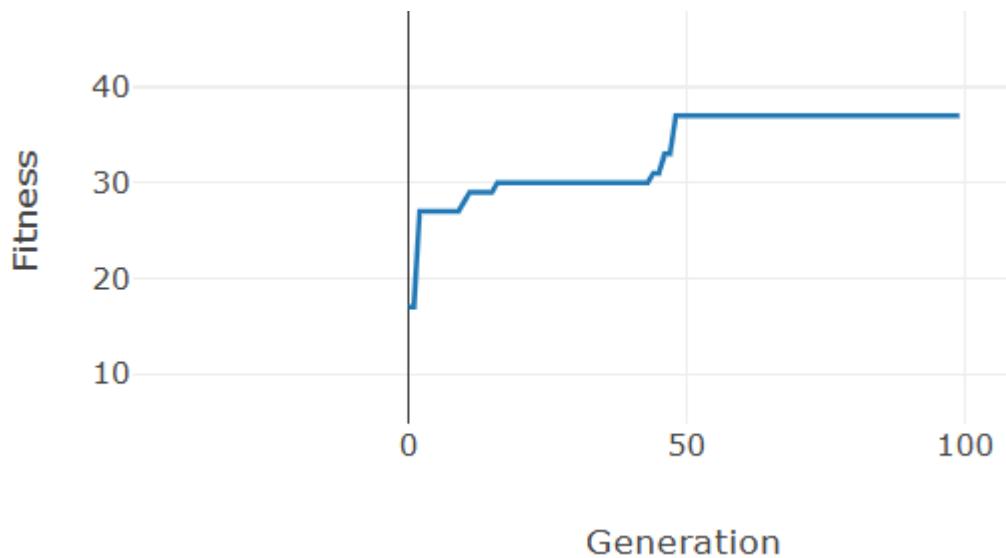


Gen. limit = 100, Mutation type = Flip-bit, Selection type = Roulette selection  
 Crossing-over type: Single-point crossover, Mutation probability = 50%

Результат: итоговое значение фитнес-функции равно 31 (выросло на 12 единиц)  
 Вывод: Roulette selection менее эффективен чем Elite selection, ликвидирует вероятность периодов стагнации, но также генерирует много отрицательных изменений и в целом менее стабилен

#### Эксперимент 9



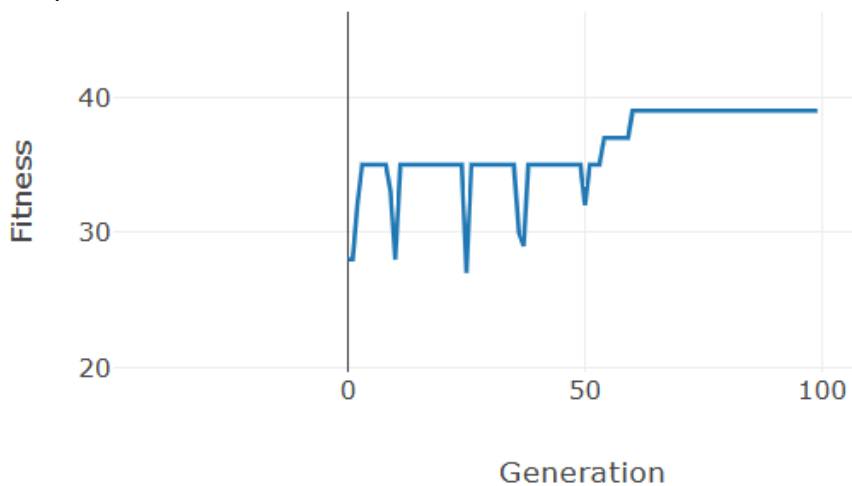


Gen. limit = 100, Mutation type = Flip-bit, Selection type = Tournament selection  
 Crossing-over type: Single-point crossover, Mutation probability = 50%

Результат: итоговое значение фитнес-функции равно 37 (выросло на 20 единиц)

Вывод: Tournament selection примерно так же эффективен как Elite selection, периоды стагнации есть, но относительно небольшие, отрицательные изменения могут возникать, но их шанс понижен

#### Эксперимент 10



Gen. limit = 100, Mutation type = Uniform, Selection type = Elite selection  
 Crossing-over type: Single-point crossover, Mutation probability = 50%

Результат: итоговое значение фитнес-функции равно 37 (выросло на 11 единиц)

Вывод: Uniform мутация порождает сильные скачки значений в отрицательную сторону - сильные отрицательные изменения - что делает ее значительно менее эффективной

## Задание 2: Муравьиный алгоритм

Код для реализации задания на Python:

---

```
import numpy as np

def ant_colony_optimization(distance_matrix, num_ants, evaporation_rate, alpha, beta,
iterations):
    num_cities = distance_matrix.shape[0]
    pheromone_matrix = np.ones((num_cities, num_cities)) # Инициализация феромонов

    best_path = None
    best_path_length = np.inf

    for iteration in range(iterations):
        all_paths = []
        all_path_lengths = []

        for ant in range(num_ants):
            current_city = np.random.randint(num_cities) # Случайно выбираем начальный
город
            path = [current_city]
            unvisited_cities = list(range(num_cities))
            unvisited_cities.remove(current_city)

            while unvisited_cities:
                probabilities = []
                for city in unvisited_cities:
                    pheromone = pheromone_matrix[current_city, city]
                    distance = distance_matrix[current_city, city]
                    probabilities.append((pheromone**alpha) * ((1/distance)**beta))

                probabilities = np.array(probabilities)
                probabilities /= probabilities.sum() # Нормализация вероятностей

                next_city = np.random.choice(unvisited_cities, p=probabilities) # Выбор
следующего города
                path.append(next_city)
                unvisited_cities.remove(next_city)
                current_city = next_city

            # Вернуться в начальный город, чтобы замкнуть маршрут
            path.append(path[0])
            all_paths.append(path)

        # Вычисление длины маршрута
        path_length = 0
        for i in range(len(path) - 1):
```

```

        path_length += distance_matrix[path[i], path[i+1]]
    all_path_lengths.append(path_length)

    # Обновление лучшего маршрута
    if path_length < best_path_length:
        best_path_length = path_length
        best_path = path

    # Испарение феромонов
    pheromone_matrix *= (1 - evaporation_rate)

    # Обновление феромонов на основе маршрутов муравьев
    for i in range(num_ants):
        path = all_paths[i]
        path_length = all_path_lengths[i]
        for j in range(len(path) - 1):
            city1 = path[j]
            city2 = path[j+1]
            pheromone_matrix[city1, city2] += 1 / path_length
            pheromone_matrix[city2, city1] += 1 / path_length # Убедитесь, что матрица
феромонов симметрична

    print(f"Итерация {iteration + 1}: Лучший маршрут = {best_path}, Длина =
{best_path_length}")

    return best_path, best_path_length

# Параметры задачи и алгоритма
num_cities = 6
num_ants = 10
evaporation_rate = 0.5
alpha = 1
beta = 2
iterations = 2

# Генерация случайной матрицы расстояний (симметричной)
distance_matrix = np.random.randint(10, 100, size=(num_cities, num_cities))
for i in range(num_cities):
    distance_matrix[i, i] = 0 # Расстояние от города до самого себя = 0
for i in range(num_cities):
    for j in range(i):
        distance_matrix[i, j] = distance_matrix[j, i] # Обеспечиваем симметричность

print("Матрица расстояний:")
print(distance_matrix)

# Запуск муравьиного алгоритма

```

```
best_path, best_path_length = ant_colony_optimization(distance_matrix, num_ants,  
evaporation_rate, alpha, beta, iterations)
```

```
print("\nЛучший найденный маршрут:", best_path)  
print("Длина лучшего маршрута:", best_path_length)
```

---

Вывод кода:

Матрица расстояний:

```
[[ 0 44 17 20 17 88]  
 [44  0 88 59 92 35]  
 [17 88  0 75 75 82]  
 [20 59 75  0 23 52]  
 [17 92 75 23  0 72]  
 [88 35 82 52 72  0]]
```

Итерация 1: Лучший маршрут = [4, 3, 1, 5, 2, 0, 4], Длина = 233

Итерация 2: Лучший маршрут = [3, 4, 0, 2, 1, 5, 3], Длина = 232

Лучший найденный маршрут: [3, 4, 0, 2, 1, 5, 3]

Длина лучшего маршрута: 232